

DAY 17

Multi-Chain Development EVM Networks + Solana Fundamentals

60-minute beginner lecture Ethereum, Base, Arbitrum, Optimism, BNB Smart Chain, Polygon and Solana - deployment portability, network differences, account/program models, SPL tokens, PDAs, Anchor and client delivery strategy.

Career focus: Learn to recognize when a blockchain outsourcing request is a simple EVM deployment/configuration task and when it requires a different architecture, toolchain and security model.

Course position: Days 1-16 built EVM, Solidity, security, Web3, backend, DeFi and oracle skills. Day 17 expands those skills into multi-chain delivery and introduces Solana as the first major non-EVM architecture in the course.

Technical update: This lecture reflects official developer documentation available in August 2026. Network parameters and tooling can change; verify production values against official documentation before deployment.

1. Learning Objectives & 60-Minute Teaching Plan

By the end of Day 17, you should be able to classify a network as EVM-compatible or non-EVM, reuse Solidity tooling across several EVM chains, identify chain-specific deployment risks, explain L1/L2/sidechain differences, and describe Solana accounts, programs, instructions, SPL tokens, PDAs and Anchor at a practical beginner level.

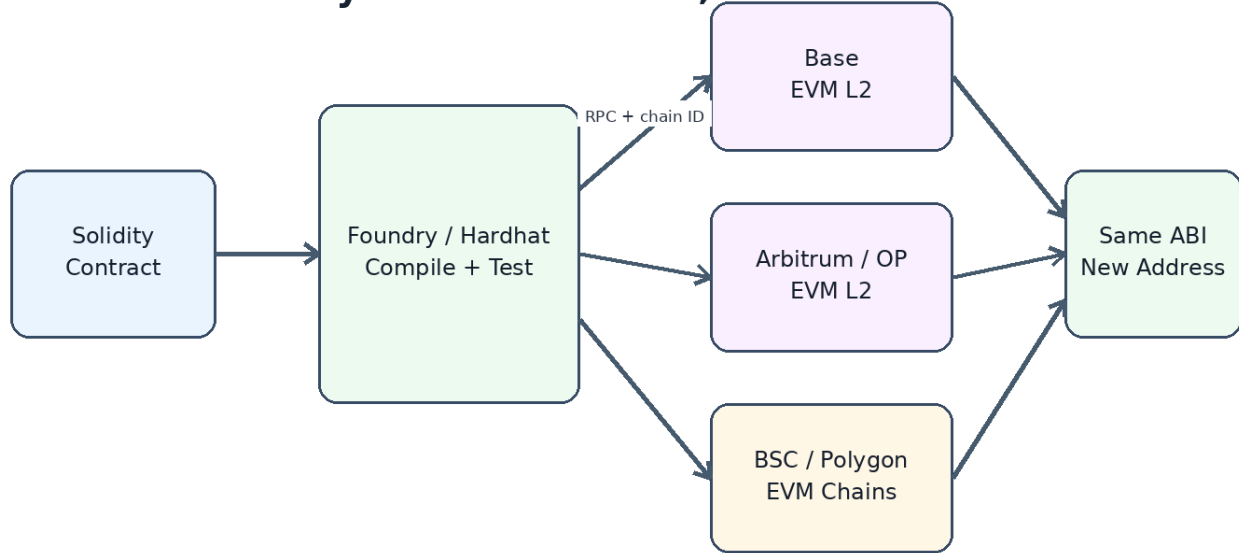
Time	Topic	Outcome
0-5 min	Multi-chain mental model	Separate portable EVM skills from architecture-specific work.
5-13 min	EVM network comparison	Compare Ethereum, Base, Arbitrum, OP Mainnet, BSC and Polygon.
13-21 min	Portable deployment workflow	Use chain IDs, RPCs, explorers, addresses and deployment config safely.
21-28 min	L2 / bridge considerations	Understand settlement, canonical assets, withdrawals and cross-chain risk.
28-34 min	EVM vs Solana	Reset the mental model before moving to non-EVM development.
34-44 min	Solana accounts + programs + PDAs	Understand state ownership, instructions and deterministic program addresses.
44-51 min	SPL tokens + Anchor	Model mints/token accounts and read an Anchor program.
51-56 min	Hands-on multi-chain lab	Plan one EVM multi-deploy and one Solana local/devnet exercise.
56-59 min	Freelancer scenarios	Scope migration, deployment and debugging jobs.
59-60 min	Review	Recite the EVM-portability and Solana-account checklists.

60-second target explanation: EVM chains reuse the same Solidity bytecode model and usually the same ABI, wallets and JSON-RPC methods, but deployment addresses, gas assets, bridges and chain-specific behavior differ. Solana is not EVM: programs are executable accounts, mutable state is stored in separate accounts, instructions explicitly declare the accounts they use, and SPL tokens use mint and token-account structures.

2. The Multi-Chain Developer Mental Model

The phrase "multi-chain" can mean two very different jobs. The first is deploying the same EVM application to several EVM-compatible networks. The second is implementing the same product idea on a fundamentally different runtime such as Solana. Treating those as the same task is one of the most expensive beginner mistakes.

EVM Portability: Same Contract, New Network



Client request	Likely engineering classification
Deploy our Solidity token on Base and Arbitrum	Mostly EVM configuration + deployment + verification + chain testing
Move our Ethereum staking app to BSC	Usually EVM port with dependency/address/gas/DEX checks
Support Polygon and Ethereum in one frontend	Multi-chain configuration + address registry + wallet chain switching
Port our Solidity staking protocol to Solana	Architecture rewrite: Rust/Anchor, accounts, PDAs, SPL tokens, new tests and client integration
Bridge our token between Ethereum and L2	Cross-chain protocol/integration task with additional security assumptions

Freelancer rule: Never quote a "multi-chain migration" before identifying whether the destination shares the EVM execution model.

3. EVM Portability: What Actually Transfers?

EVM-compatible chains execute Ethereum-style bytecode and expose familiar Ethereum development interfaces. This makes Solidity expertise highly reusable, but "compatible" does not mean every operational detail is identical.

Usually portable	Must still be re-validated per chain
Solidity source code	RPC endpoint and chain ID
ABI encoding and contract calls	Gas token and fee behavior
Foundry / Hardhat workflows	Deployed contract addresses
ethers.js / viem contract interface	Block time / confirmation assumptions
OpenZeppelin libraries	Predeploys, native system contracts and protocol differences
ERC-20 / ERC-721 standards	DEX/router/oracle/bridge addresses
MetaMask-style EVM wallet flow	Explorer verification support
JSON-RPC core methods	Finality, sequencing, reorg and withdrawal semantics

```
// The ABI can remain the same while the deployment address changes.
```

```
const deployments = {
  8453: { name: "Base",    app: "0xBASE..." },
  42161: { name: "Arbitrum", app: "0xARB..." },
  10:   { name: "Optimism", app: "0xOP..." },
  56:   { name: "BSC",     app: "0xBSC..." },
  137:  { name: "Polygon",  app: "0xPOLY..." }
};
```

4. 2026 EVM Network Comparison for Freelancers

Network	Type / relationship	Mainnet chain ID	Typical gas asset	Beginner developer note
Ethereum	L1 / settlement layer	1	ETH	Baseline EVM behavior; highest familiarity and deepest tooling.
Base	EVM-compatible Ethereum L2	8453	ETH	Uses EVM tooling; Base Sepolia = 84532.
Arbitrum One	EVM-compatible Ethereum L2	42161	ETH	Solidity contracts deploy with familiar tooling; Sepolia = 421614.
OP Mainnet	OP Stack Ethereum L2	10	ETH	Designed to be EVM equivalent; OP Sepolia = 11155420.
BNB Smart Chain	EVM-compatible chain	56	BNB	Ethereum tooling transfers; BSC Testnet = 97.
Polygon Chain / PoS	EVM-compatible chain anchored to Ethereum	137	POL	Amoy testnet = 80002; check current bridge/token mapping assumptions.

Important terminology: Base, Arbitrum One and OP Mainnet are Ethereum L2 systems. BNB Smart Chain is not an Ethereum L2. Polygon PoS/Polygon Chain is EVM-compatible and anchored/checkpointed to Ethereum, but its security/consensus model is not identical to an Ethereum rollout.

Network facts above are included for learning. For production deployment, always confirm the current official network reference because public RPC endpoints, fee mechanisms, testnets and protocol features can change.

5. Ethereum as the Baseline

When a client says "Ethereum-compatible," your baseline assumptions come from Ethereum: EVM bytecode, EOAs/contracts, chain IDs, nonce-based signed transactions, gas, ABI, events/logs and JSON-RPC. Other EVM networks intentionally preserve much of this developer experience.

- Compile once, but treat each deployed network as a separate environment.
- Do not reuse a mainnet address on another chain unless the protocol explicitly guarantees deterministic deployment there.
- Token symbols are not identities. USDC on two networks may use different contract addresses and bridge provenance.
- Block confirmation/finality policy should be configured per network and per business risk.
- A frontend should detect the connected chain and refuse or switch when the user is on an unsupported network.

5.1 Chain ID is a security boundary

Chain ID helps distinguish networks and is incorporated into replay-protected EVM transaction signing. A frontend/backend that ignores chain ID can send a user to the wrong deployment or display incorrect balances.

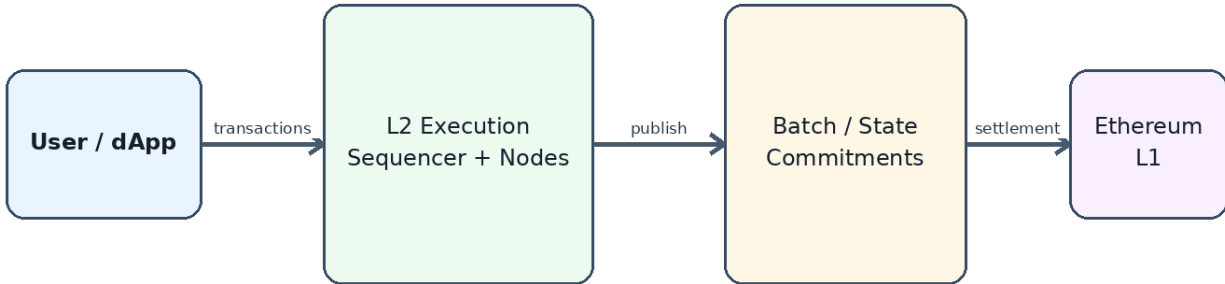
```
const network = await provider.getNetwork();
const chainId = Number(network.chainId);

if (!deployments[chainId]) {
  throw new Error(`Unsupported chain: ${chainId}`);
}
```

6. Ethereum L2s: Base, Arbitrum and OP Mainnet

Layer-2 networks execute transactions away from Ethereum L1 and publish data/state commitments back to Ethereum according to their protocol. For application developers, the UX feels EVM-like, but settlement, sequencing, fees and bridging introduce additional concepts.

L2 Mental Model: Execute, Publish, Settle



Question	Why the developer must care
Where does execution happen?	Your contract executes on the L2, not directly on Ethereum mainnet.
What is the gas asset?	Many standard Ethereum L2s use ETH, but do not generalize blindly to every custom chain.
How are L1/L2 messages delivered?	Bridge/message contracts and delays affect cross-layer workflows.
What does "confirmed" mean?	Sequencer inclusion, L2 block confirmation and stronger settlement/finality are different concepts.
What happens during sequencer issues?	Price-sensitive and time-sensitive apps may require special safeguards.
How long do withdrawals take?	Canonical bridge withdrawal behavior depends on the L2 protocol.

7. Base: What Transfers and What to Check

Base documents itself as an EVM-compatible Layer 2 network. Existing Ethereum contracts and tooling generally transfer, but you still need Base-specific network configuration, deployed addresses and operational testing.

Item	Base Mainnet	Base Sepolia
Chain ID	8453	84532
Gas asset	ETH	ETH
Developer model	EVM / Ethereum JSON-RPC	EVM / Ethereum JSON-RPC
Recommended usage	Production	Testing

- Use the official network reference or a production RPC provider rather than hard-coding a random public endpoint.
- Keep per-chain contract addresses in configuration, not buried throughout frontend code.
- Verify contracts on the chain explorer after deployment.
- Test wallet switching, wrong-chain UX and event indexing on Base separately.
- Base may expose newer chain-specific features; do not assume every feature exists on generic EVM chains.

```
[rpc_endpoints]
base_sepolia = "${BASE_SEPOLIA_RPC_URL}"

[profile.default]
# Solidity compilation settings remain shared.
```

8. Arbitrum: Solidity Portability + Nitro/L2 Context

Arbitrum documents Solidity support as Ethereum-compatible: Solidity contracts compile to EVM bytecode and can be deployed with familiar Ethereum tooling. The important freelancer step is validating protocol dependencies and L2 behavior rather than rewriting ordinary Solidity.

Item	Arbitrum One	Arbitrum Sepolia
Chain ID	42161	421614
Gas asset	ETH	ETH
Contract language	Solidity/EVM compatible	Solidity/EVM compatible
Use	Production	Testing

8.1 Common migration checklist

1. Deploy the same tested bytecode or rebuild from the same commit and compiler settings.
2. Replace Ethereum-only dependency addresses with Arbitrum equivalents.
3. Verify oracle, DEX, bridge and token contract addresses.
4. Test event indexing and RPC behavior with the target provider.
5. Define confirmation/finality policy for deposits, withdrawals and business actions.
6. Verify contracts and publish deployment metadata.

Client communication: "Solidity-compatible" does not mean a third-party protocol address from Ethereum will magically work on Arbitrum. Address/config migration is part of the job.

9. OP Mainnet / OP Stack: EVM Equivalent, Not "Assume Identical"

OP Stack documentation describes standard OP Stack chains as EVM equivalent with a small set of differences from Ethereum. This is intentionally developer-friendly: existing Solidity applications usually need only limited chain-specific adaptation.

Concept	Freelancer interpretation
EVM equivalent	Ethereum development patterns largely transfer.
OP Stack	Modular stack used to build L2 chains such as OP Mainnet.
Sequencer	Orders L2 transactions before data/state is published to L1.
Batcher / proposer / challenger	Protocol infrastructure; usually not application code, but relevant to settlement/security understanding.
L1 data component of fees	Do not estimate user cost from EVM gas used alone.
Bridging	Cross-domain messaging/asset flows require dedicated contracts and timing expectations.

```
// OP Mainnet / OP Sepolia chain IDs
const OP_MAINNET = 10;
const OP_SEPOLIA = 11155420;
```

10. BNB Smart Chain: Familiar EVM, Different Ecosystem

BNB Smart Chain is EVM-compatible and exposes Ethereum-style JSON-RPC APIs, so Solidity, MetaMask, Hardhat, Foundry and ethers.js skills transfer well. The ecosystem, gas asset, infrastructure and protocol addresses are different.

Item	BSC Mainnet	BSC Testnet
Chain ID	56	97
Gas asset	BNB	tBNB
EVM compatibility	Yes	Yes
Typical explorer	BscScan	BscScan testnet

10.1 Porting mistakes to avoid

- Using an Ethereum router/oracle address on BSC.
- Assuming token contracts are the same because symbols match.
- Hard-coding Ethereum chain ID 1 in signatures or frontend guards.
- Using a public RPC for production without rate-limit/reliability planning.
- Ignoring chain-specific finality/validator assumptions when defining business confirmations.

11. Polygon Chain / PoS: EVM Portability with Its Own Architecture

Polygon developer documentation describes Polygon Chain / PoS as EVM-compatible and anchored to Ethereum. Solidity contracts can generally be deployed without source-code changes, but RPC, chain ID, gas token, bridge/token mapping and network behavior are separate concerns.

Item	Polygon Mainnet	Amoy Testnet
Chain ID	137	80002
Gas token in current RPC reference	POL	POL
Execution model	EVM-compatible	EVM-compatible
Parent/anchor context	Ethereum	Sepolia

2026 documentation note: Polygon documentation has evolved and some older pages may still use legacy MATIC wording. Use the current network/RPC reference and current bridge/token documentation when delivering production work.

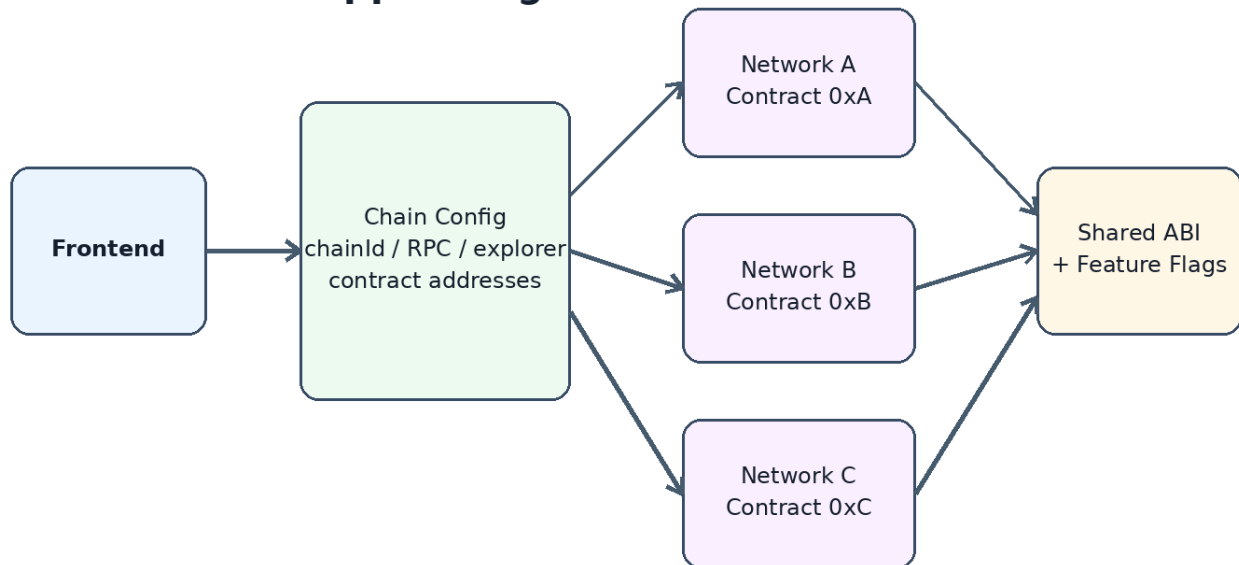
11.1 What to test

- Contract deployment and verification
- POL gas funding
- Token address/mapping
- Bridge deposits and withdrawals if used
- RPC/websocket provider behavior
- Frontend chain switch to 137/80002
- Indexer and confirmation rules

12. Professional Multi-Chain Configuration

Do not scatter if/else statements throughout your application. Centralize chain metadata, RPC configuration, deployment addresses and feature availability.

Multi-Chain dApp Configuration Pattern



```

export const CHAINS = {
  84532: {
    name: "Base Sepolia",
    rpcEnv: "BASE_SEPOLIA_RPC_URL",
    explorer: "https://sepolia.basescan.org",
    contracts: { staking: "0x..." }
  },
  421614: {
    name: "Arbitrum Sepolia",
    rpcEnv: "ARB_SEPOLIA_RPC_URL",
    explorer: "https://sepolia.arbiscan.io",
    contracts: { staking: "0x..." }
  }
};
  
```

12.1 Store deployment metadata

```

deployments/
  base-sepolia.json
  arbitrum-sepolia.json
  optimism-sepolia.json
  
```

```

# Each file: chainId, commit, compiler, contract address,
# constructor args, deployment tx, block number, verifier URL.
  
```

13. Multi-Chain Foundry Deployment Pattern

A simple deployment script should receive configuration rather than embedding one network. The same script can then be reused against different RPC aliases.

```
// script/Deploy.s.sol
pragma solidity ^0.8.24;

import {Script} from "forge-std/Script.sol";
import {FreelancerToken} from "../src/FreelancerToken.sol";

contract Deploy is Script {
    function run() external returns (FreelancerToken token) {
        uint256 key = vm.envUint("DEPLOYER_PRIVATE_KEY");
        vm.startBroadcast(key);
        token = new FreelancerToken();
        vm.stopBroadcast();
    }
}

forge script script/Deploy.s.sol:Deploy \
  --rpc-url base_sepolia \
  --broadcast

forge script script/Deploy.s.sol:Deploy \
  --rpc-url arbitrum_sepolia \
  --broadcast
```

Security: Never place production private keys in the repository. Prefer secure signer infrastructure, hardware wallets, multisig-controlled ownership and separated deployer/admin roles for serious client deployments.

14. Same Source Does Not Mean Same Deployment Result

Even when Solidity source is identical, external conditions can differ. Treat each chain as its own release target.

Check	Example
Constructor/config parameters	Different oracle or router addresses
CREATE address	Depends on deployer + nonce
CREATE2 address	Can be deterministic only if deployer/factory/salt/init code match
External protocol dependency	May not exist on destination chain
Native token assumptions	ETH vs BNB vs POL
Fee assumptions	L2 may include L1 data cost or chain-specific fee behavior
Block/time assumptions	Different block production and finality behavior
Precompiles/system contracts	May differ across chains
Explorer verification	Different verifier/explorer endpoints

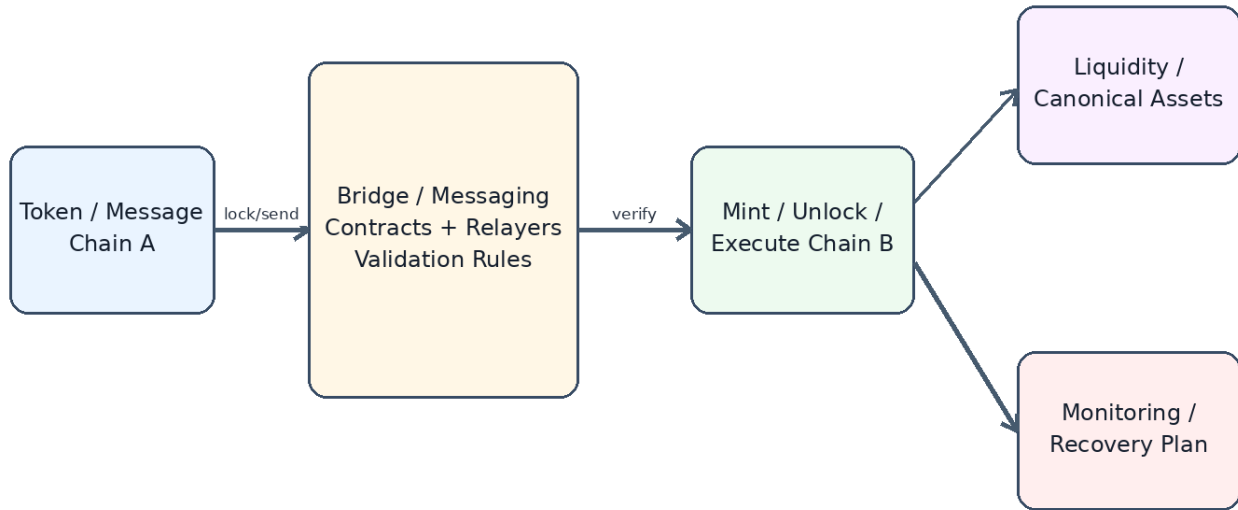
14.1 Release checklist

1. Run unit tests locally.
2. Run chain-specific fork/integration tests where practical.
3. Deploy to destination testnet.
4. Verify source.
5. Run scripted smoke tests.
6. Record addresses and transaction hashes.
7. Test frontend/backend against exact deployment.
8. Complete owner/admin handoff only after verification.

15. Bridges and Cross-Chain Messaging

A bridge or messaging protocol adds a new security boundary. Your application now depends on contracts and validation rules on more than one network, plus relaying/finality logic between them.

Bridge Flow: More Validation, More Risk



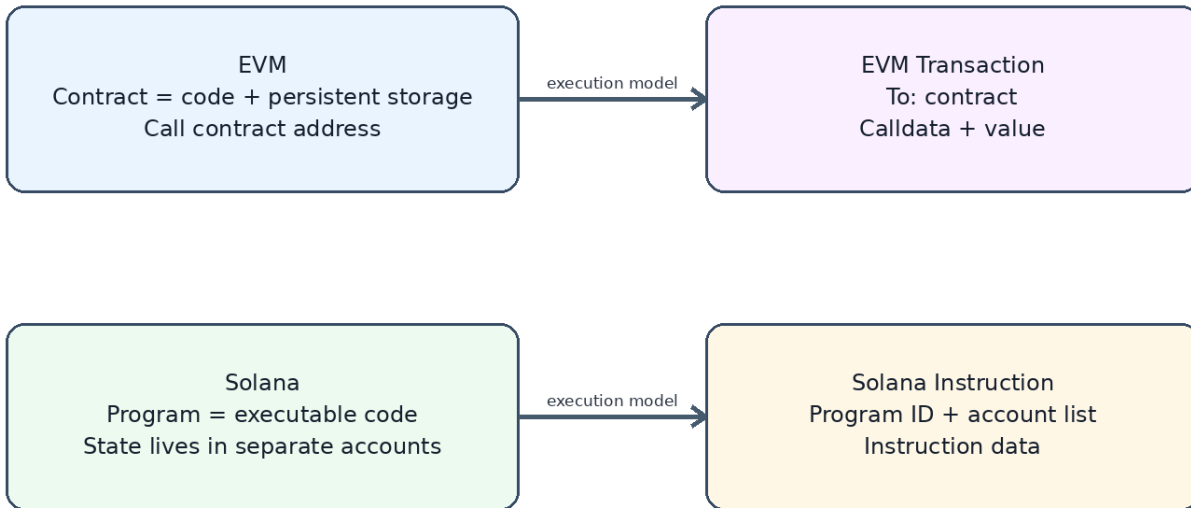
Risk	Question to ask before coding
Asset identity	Is the destination token canonical/native, bridge-wrapped or third-party issued?
Finality	When is the source message considered safe to act on?
Replay	How is the same cross-chain message prevented from executing twice?
Message authentication	Who/what proves a source-chain event happened?
Liquidity	Does the bridge lock/mint, burn/mint or use liquidity pools?
Failure/recovery	What happens if relay/execution fails after source action?
Admin/security	Who can upgrade/pause bridge contracts?
Rate limits	Can bridge exposure be capped during an incident?

Scope warning: "Add bridging" is not a UI-only feature. It changes protocol risk, token identity, settlement timing, monitoring and incident response.

16. Reset the Mental Model: EVM vs Solana

Solana is the point where you must stop thinking "change RPC and chain ID." Solana uses a different runtime, transaction/account model, programming environment and token architecture.

EVM vs Solana: Different State Architecture

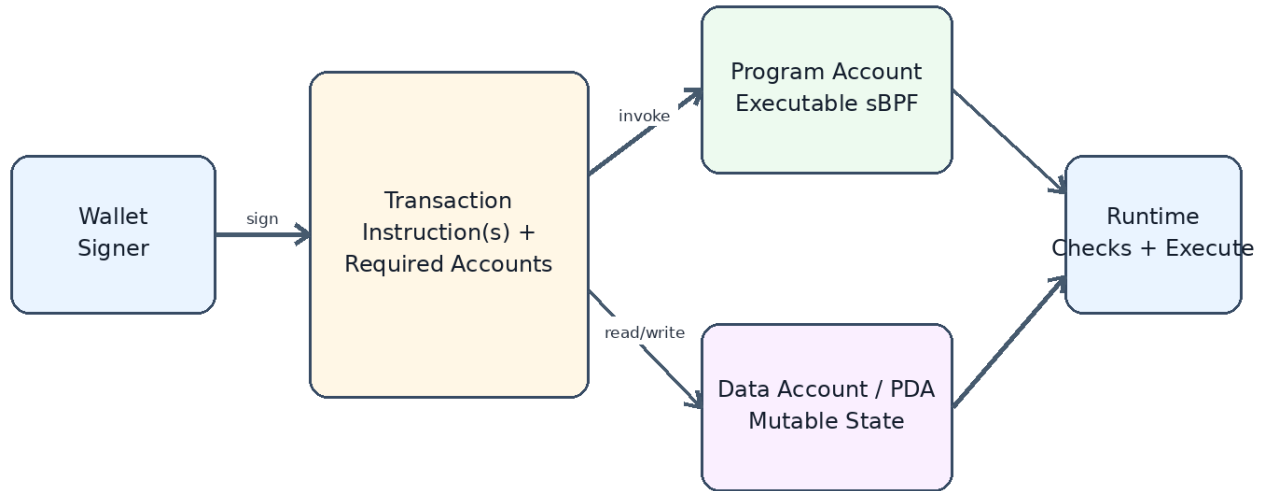


EVM concept	Solana concept
Contract address	Program ID for executable program
Contract persistent storage	Separate data accounts owned by programs
msg.sender	Signer/writable account constraints passed to instruction
Calldata function selector	Instruction discriminator/data + account list
Gas	Transaction fee + compute-unit budget model
ERC-20 contract	SPL Token Program + Mint account + token accounts
Contract-created deterministic address	PDA derived from seeds + program ID
Solidity	Rust commonly, often with Anchor framework
ABI	IDL commonly used by Anchor clients

17. Solana Accounts: State Is Data

Solana documentation defines an account as the fundamental unit for storing state. Every account has a 32-byte address and fields such as lamports, data, owner and executable status. Programs are accounts too, but mutable application state lives in separate data accounts.

Solana Program + Account Model



Account idea	Meaning
System account	Wallet-like account owned by the System Program; can hold SOL.
Program account	Executable sBPF code; executable flag is true.
Data account	Stores application state; owned by a program.
Token Mint account	Defines a token and shared mint state.
Token account / ATA	Holds units of one mint for an owner.
PDA account	Deterministic address controlled by a program, not by a private key.
Sysvar	Cluster/runtime state exposed at predefined addresses.

Critical difference: A Solana program is described as stateless in the application sense: it modifies state in accounts passed to its instructions. If a required account is not provided, the program cannot simply look up arbitrary mutable state the way an EVM contract reads its own storage.

18. Solana Transactions and Instructions

A Solana transaction contains one or more instructions. Each instruction identifies the program to execute, the accounts it needs and encoded instruction data. Transactions are atomic: if one instruction fails, the transaction does not partially commit successful earlier instructions.

```
Instruction {
  program_id: <program address>,
  accounts: [
    { pubkey: user, is_signer: true, is_writable: true },
    { pubkey: vault, is_signer: false, is_writable: true }
  ],
  data: <encoded instruction bytes>
}
```

Account flag	Why it matters
Signer	Runtime verifies the transaction includes authorization from that key.
Writable	Program may modify lamports/data for that account during execution.
Read-only	Program may inspect but not modify.
Owner	Runtime/program uses ownership to determine which program can modify account data.

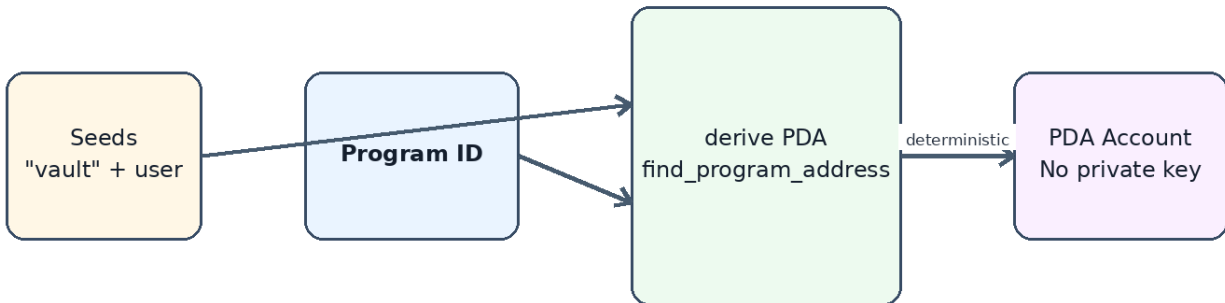
18.1 Cross-Program Invocation (CPI)

Programs can invoke other programs, similar to composing protocols on EVM. CPI security requires validating program IDs, account ownership, signer expectations and PDA seeds rather than trusting arbitrary accounts supplied by a caller.

19. Program Derived Addresses (PDAs)

A PDA is deterministically derived from a program ID and one or more seeds. Solana documentation notes that PDAs are deliberately off the Ed25519 curve, so no private key exists for them. The owning program can authorize PDA actions through signed program invocation mechanisms.

PDA: Deterministic Program-Controlled Address



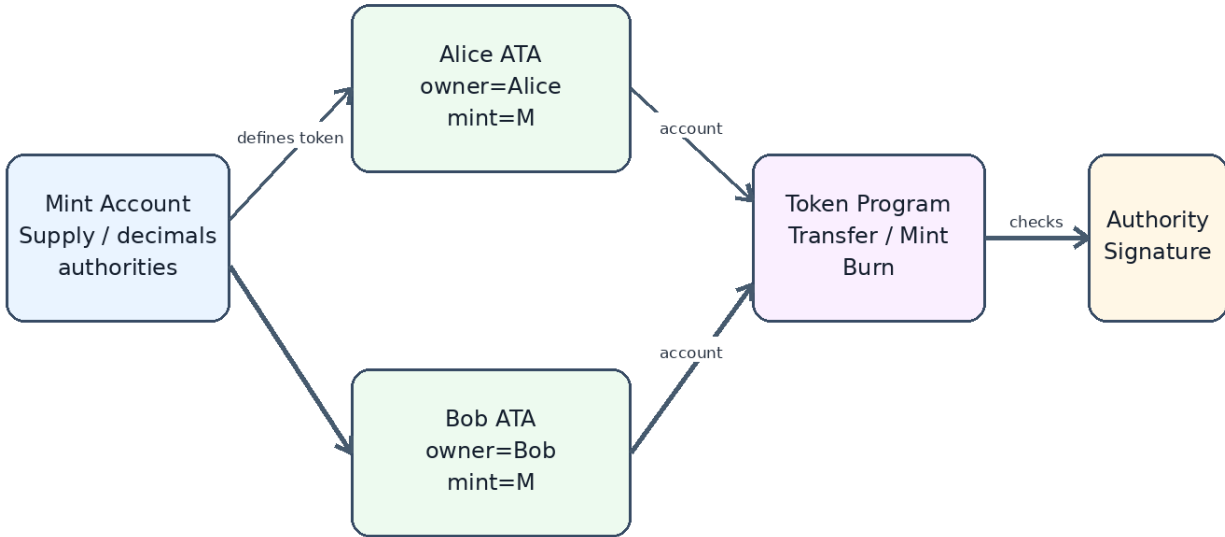
```
// Anchor account constraint concept
#[account(
  seeds = [b"profile", user.key().as_ref()],
  bump
)]
pub profile: Account<'info, Profile>
```

- Use PDAs for deterministic program state such as user profiles, vault authorities or configuration accounts.
- Validate the expected seeds and bump; do not accept an arbitrary account merely because the client calls it a vault.
- A PDA is an address/authority mechanism, not automatically a data account - it must be created/initialized when state is needed.
- PDA design affects discoverability, uniqueness and authorization throughout the protocol.

20. SPL Tokens: Mint Account + Token Accounts

Solana tokens are commonly implemented using the SPL Token Program or Token Extensions Program. The token itself is identified by a Mint account; user balances live in token accounts, commonly Associated Token Accounts (ATAs).

SPL Token Model: Mint + Token Accounts



EVM ERC-20	Solana SPL token model
Token contract stores balances	Token Program operates on Mint + token-account state
balanceOf(user)	Read user token account / ATA amount
totalSupply()	Mint account stores supply
mint() role inside token contract	Mint authority authorizes mint instruction
approve/allowance	Token account can configure delegate/allowance
burn()	Burn instruction reduces account amount and mint supply
Wallet address can directly hold ERC-20 balance	Wallet owns one or more token accounts for a mint

```

# Devnet learning workflow (illustrative)
solana config set --url devnet
solana airdrop 1

spl-token create-token
spl-token create-account <MINT>
spl-token mint <MINT> 1000
spl-token accounts
  
```

21. Associated Token Accounts and Authorities

An Associated Token Account is the default deterministic token account for a wallet + mint pair. It is derived by the Associated Token Program, which makes token-account discovery easier for wallets and applications.

Authority	Controls
Mint authority	Creation of new token units for the mint.
Freeze authority	Freeze/thaw token accounts when configured.
Token-account owner	Transfers/burns and ownership-level token operations.
Delegate	Limited authority to transfer/burn up to an approved amount.
Close authority	May close a token account when configured/allowed.

Security lesson: "I have the wallet private key" is not enough information to answer every Solana token question. You must identify which account is involved and which authority role controls the requested operation.

21.1 Token Extensions

The Token Extensions Program (often called Token-2022) adds optional features such as transfer fees, metadata pointer, non-transferable tokens, pausable mint and other extension-specific behavior. Many extensions must be chosen during initialization and cannot simply be added later, so requirements should be clarified before mint creation.

22. Anchor Framework: Why Freelancers Use It

Anchor is a framework for writing Solana programs in Rust with macros and conventions that reduce boilerplate and make account validation clearer. It also generates an IDL used by clients.

Anchor concept	Purpose
<code>#[program]</code>	Declares instruction handlers.
<code>#[derive(Accounts)]</code>	Defines and validates accounts required by an instruction.
<code>#[account]</code>	Declares program account data types / constraints.
<code>Signer<'info></code>	Requires a signer account.
<code>Account<'info, T></code>	Deserializes/validates an account into a typed structure.
<code>seeds + bump</code>	Validates a PDA.
<code>System / Token interfaces</code>	Makes CPI integrations easier to express.
<code>IDL</code>	Machine-readable client interface for instructions/accounts/types.

```
use anchor_lang::prelude::*;

declare_id!("Fg6PaFpoGXkYsidMpWxTWqkZEr4YxZMnS5Q5kqbdP6W"); // example only

#[program]
pub mod freelancer_profile {
    use super::*;

    pub fn initialize(ctx: Context<Initialize>, name: String) -> Result<()> {
        let profile = &mut ctx.accounts.profile;
        profile.owner = ctx.accounts.user.key();
        profile.name = name;
        Ok(())
    }
}
```

23. Anchor Account Constraints: Where Security Lives

```
#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(mut)]
    pub user: Signer<'info>,

    #[account(
        init,
        payer = user,
        space = 8 + Profile::INIT_SPACE,
        seeds = [b"profile", user.key().as_ref()],
        bump
    )]
    pub profile: Account<'info, Profile>,

    pub system_program: Program<'info, System>,
}

#[account]
#[derive(InitSpace)]
pub struct Profile {
    pub owner: Pubkey,
    #[max_len(64)]
    pub name: String,
}
```

Read the constraints as security requirements: the user must sign, the profile address must be the expected PDA, the user pays for creation, the System Program is supplied, and the account has defined storage space.

Bad assumption	Correct review question
"The frontend sends the right account"	Does the program itself constrain/validate the account?
"The user address is in instruction data"	Is a Signer account required and compared where needed?
"This is our vault"	Is the vault address derived/validated by expected PDA seeds?
"This token account belongs to our mint"	Are mint/owner/token-program constraints checked?
"This program ID is trusted"	Is the CPI target fixed/validated?

24. Solana Client Integration: IDL + Program + Accounts

An Anchor client uses the program ID/IDL and supplies the accounts required by each instruction. Client libraries evolve, so verify the Anchor client and Solana web3 library compatibility for the project version before upgrading dependencies.

```
// Conceptual Anchor TypeScript flow
const program = new Program(idl, provider);

const [profilePda] = PublicKey.findProgramAddressSync(
  [Buffer.from("profile"), wallet.publicKey.toBuffer()],
  program.programId
);

await program.methods
  .initialize("Alice")
  .accounts({
    user: wallet.publicKey,
    profile: profilePda,
    systemProgram: SystemProgram.programId
  })
  .rpc();
```

24.1 EVM vs Solana frontend difference

EVM frontend	Solana frontend
contract address + ABI	program ID + IDL
function call determines storage inside contract	instruction declares program + required accounts
wallet signer signs transaction	wallet signer signs Solana transaction
events/logs indexed by EVM log model	program logs/account changes use Solana RPC subscriptions/indexers
chainId selects network	cluster/RPC/genesis context selects Solana network

25. Solana Testing & Deployment Workflow

Use local simulation/testing first, then Devnet, then production only after authority and upgrade decisions are explicit. Solana public RPC endpoints are documented as rate-limited and not intended for production workloads.

```
# Typical Anchor workflow
anchor build
anchor test

# Select devnet for deployment testing
solana config set --url devnet
solana balance
anchor deploy
```

Stage	What to verify
Unit/program tests	Instruction success + failure cases, account constraints, PDA seeds, token authority logic
Local validator / simulation	Transaction/account behavior without public-network dependency
Devnet	Wallet/client integration, account creation, rent/fees, deployment process
Program verification	Reproducible/verified build process where required
Mainnet	Upgrade authority, admin keys, monitoring, RPC capacity, incident plan

Upgrade authority: A deployed Solana program may remain upgradeable when an upgrade authority exists. Revoking it can make the program immutable. This is a governance/security choice that must be documented for the client.

26. Hands-On Lab: Part A - Deploy One Solidity Contract to Two EVM Testnets

Use a small contract already tested in Days 5-10. The goal is not new Solidity; the goal is disciplined multi-environment delivery.

1. Choose two testnets, for example Base Sepolia and Arbitrum Sepolia.
2. Create separate RPC environment variables.
3. Deploy from the same source commit and compiler configuration.
4. Record each chain ID, deployment address, transaction hash and block number.
5. Verify the contract on each explorer.
6. Call one view function and one state-changing function on each chain.
7. Capture the same event on both networks.
8. Update a frontend config map and prove that the UI switches addresses by chain ID.

Evidence	Base Sepolia	Arbitrum Sepolia
Chain ID	84532	421614
Contract address	_____	_____
Deploy tx	_____	_____
Verified?	Yes / No	Yes / No
Smoke test tx	_____	_____

27. Hands-On Lab: Part B - Solana Program & Token Investigation

The Solana half is deliberately investigative on Day 17. The goal is to become fluent in the account model before trying to build a large program.

1. Configure Solana Devnet and obtain test SOL.
2. Inspect your wallet/system account.
3. Create or inspect an SPL token mint.
4. Create the associated token account for your wallet.
5. Mint test units if you control mint authority.
6. Identify mint authority, freeze authority, decimals and supply.
7. Derive a sample PDA from known seeds/program ID in a small script or Anchor exercise.
8. Open an Anchor example and identify the program handler, Accounts struct, signer, PDA constraints and state account.

Do not use real assets for this lab: Use localnet/devnet and disposable learning keys. Never paste a production seed phrase or private key into classroom code, screenshots, repositories or AI tools.

28. How to Choose a Chain for a Client

A professional answer is not "Chain X is best." Choose based on product requirements, ecosystem dependencies, security model, liquidity, user wallets, cost, latency, developer tooling and operational needs.

Requirement	Questions
Existing contracts	Is the current system Solidity/EVM?
Users	Which wallets/chains do target users already use?
Liquidity/DeFi	Are required DEX/lending/oracle protocols available?
Settlement/security	Does the client require Ethereum settlement or another trust model?
Transaction cost	What operations occur frequently and who pays gas?
Latency/UX	How fast must the UI show likely/strong confirmation?
Tokens/assets	Where are the canonical assets issued?
Bridging	Will users need cross-chain asset/message flows?
Backend/indexing	Are reliable RPC/indexer providers available?
Team skills	Can the team maintain Solidity/EVM, Rust/Solana or both?

28.1 Proposal language

Recommend a short discovery milestone first when network choice materially changes architecture. The deliverable can include a dependency matrix, prototype transaction, cost/latency measurements, security assumptions and a migration estimate.

29. Multi-Chain Security Checklist

Area	EVM multi-chain check	Solana check
Network identity	Validate chain ID	Validate cluster/RPC/genesis context
Contract/program identity	Correct deployment address + bytecode	Correct program ID + verified build where needed
State identity	Correct contract storage / external addresses	Correct account owner + PDA seeds + discriminators
Tokens	Correct token contract + decimals	Correct Mint + token program + ATA/mint authority
Authorization	Owner/roles/multisig	Signer + account constraints + authorities
Cross-chain	Bridge/message validation	Bridge/message validation if used
External calls	Validate target contracts	Validate CPI target programs/accounts
Upgrades	Proxy admin/storage layout	Program upgrade authority
RPC	Rate limits, websocket, archive needs	Rate limits, subscriptions, account query needs
Monitoring	Events, receipts, reorg/finality policy	Logs, signatures, account state, commitment policy

Security mindset: Multi-chain work multiplies configuration risk. Many real failures are not sophisticated exploits - they are wrong addresses, wrong networks, wrong authorities, wrong token identities or unsafe upgrade/admin handoffs.

30. Freelancer / Upwork Scenarios

Client request	How to respond professionally
"Deploy our token to Base, BSC and Polygon"	Ask for source repo, compiler/tooling, owner/admin model, supply rules, verification requirements, destination testnets/mainnets, and any DEX/bridge integrations. Quote separate testnet + mainnet milestones.
"Our dApp works on Ethereum but fails on Arbitrum"	Check chain ID, deployment address, RPC, token/protocol addresses, fee assumptions, wallet network, revert data and explorer trace before rewriting Solidity.
"Make the same staking app on Solana"	Explain that this is a new program/account architecture, not a Solidity redeploy. Scope Rust/Anchor program, PDAs/state, SPL token accounts, tests, client integration and authority/security design.
"User sent tokens but balance is zero"	Confirm network + token identity. On EVM inspect token contract and Transfer logs; on Solana inspect Mint and token/associated token accounts.
"We have Solana upgrade authority; can we change behavior?"	Verify the exact program account/deployment state and authority first; owning an unrelated account is not equivalent to program upgrade authority.

30.1 Deliverables clients value

- Network/dependency matrix
- Deployment scripts and environment template
- Verified addresses and release manifest
- Smoke/integration tests per chain
- Frontend chain configuration
- Authority/admin handoff documentation
- Monitoring and rollback/incident notes

31. Common Debugging Matrix

Symptom	Likely checks
EVM "contract not found"	Wrong chain, wrong address, deployment failed, provider on different chain
EVM call reverts only on one chain	Different dependency/config/address, protocol-specific behavior, missing token balance/approval
Frontend shows wrong balance	Wrong chain ID or token address/decimals
Bridge deposit succeeded but destination missing	Bridge finality/relay/execution, destination token mapping, message status
Solana "account not initialized"	Wrong PDA/account, initialization not executed, incorrect program ID/seeds
Solana owner mismatch	Data/token account owned by different program than expected
SPL transfer fails	Wrong mint, wrong token program, authority/signature issue, frozen account, insufficient amount
Anchor constraint error	Signer/PDA/mint/owner/address constraint failed
Program upgrade fails	Wrong upgrade authority, loader/deployment mismatch, program immutable or insufficient buffer/funds

32. Interview Questions - Day 17

1. What does EVM-compatible mean for a Solidity developer?
2. Why can the same ABI be used on several EVM chains while contract addresses differ?
3. What is chain ID and why should a frontend validate it?
4. What is the difference between an Ethereum L2 and BNB Smart Chain?
5. What does a bridge add to the security model?
6. Why is porting Solidity to Solana not a simple redeployment?
7. Where does mutable state live in Solana?
8. What is a Solana instruction?
9. What is a PDA and why does it have no private key?
10. What is the difference between an SPL Mint and a token account?
11. What is an Associated Token Account?
12. What does Anchor `#[derive(Accounts)]` help validate?
13. What does program upgrade authority mean on Solana?
14. How would you scope a client request to support three EVM chains plus Solana?

33. Day 17 Quiz

1. True or false: If a contract works on Ethereum, every dependency address is automatically valid on Base.
2. Which chain ID identifies Base mainnet?
3. Which chain ID identifies Arbitrum One?
4. What is the main gas asset on BNB Smart Chain?
5. Name two operational differences you must still test on EVM-compatible networks.
6. Where does application state usually live in Solana?
7. What three major pieces identify a Solana instruction?
8. Why can no wallet private key directly sign as a PDA?
9. What does an SPL Mint account store conceptually?
10. What is an ATA?
11. Why is a cross-chain bridge a new security boundary?
12. What is the first scoping question when a client asks to "port to another blockchain"?

Answer key

1. False - code may port, but addresses/dependencies/configuration are chain-specific.
2. 8453.
3. 42161.
4. BNB.
5. Examples: RPC/chain ID, addresses, fees, finality/confirmations, bridge behavior, external protocol support.
6. In separate data accounts owned/controlled by programs.
7. Program ID, required account metadata/list, and instruction data.
8. PDAs are intentionally derived off the Ed25519 curve and have no private key; the program authorizes them through program-signing semantics.
9. Token identity/shared state such as supply, decimals and authorities.
10. The deterministic default token account for an owner + mint, created/derived through the Associated Token Program.
11. It adds cross-network validation, contracts, relaying/finality and asset-identity assumptions.
12. Whether the destination is EVM-compatible or requires a different runtime/architecture.

34. Homework & Portfolio Assignment

Homework A - EVM deployment matrix

Create a one-page deployment matrix for Ethereum Sepolia, Base Sepolia, Arbitrum Sepolia, OP Sepolia, BSC Testnet and Polygon Amoy. Include chain ID, gas token, RPC source, explorer, your contract address, deployment transaction and verification status. Do not include private RPC keys or signing keys in the submitted file.

Homework B - Solana account model

Draw a diagram for a staking-style Solana program showing: user signer, staking program, configuration PDA, user-position PDA, stake Mint, user ATA, vault ATA and Token Program. For each account mark signer/read-only/writable and who owns the account data.

Portfolio project

Build a "Multi-Chain Deployment Starter" repository: one Solidity contract deployed to at least two EVM testnets, shared ABI, typed chain config, deploy scripts, smoke test, verified explorer links, README architecture diagram, plus a short Solana folder that demonstrates PDA derivation and SPL Mint/ATA inspection on Devnet.

Client-style acceptance criteria

- No secrets committed
- Addresses are network-scoped
- Wrong-network UI is handled
- Deployments are reproducible
- Test transactions are documented
- Solana examples identify authority/account ownership correctly

35. Day 17 Cheat Sheet

Topic	Remember
EVM portability	Same Solidity/ABI/tooling often transfers; addresses/config/dependencies do not.
Base	EVM-compatible Ethereum L2; 8453 mainnet / 84532 Sepolia.
Arbitrum One	EVM-compatible Ethereum L2; 42161 / 421614 Sepolia.
OP Mainnet	OP Stack EVM-equivalent L2; 10 / 11155420 Sepolia.
BSC	EVM-compatible; 56 / 97 testnet; gas = BNB.
Polygon	EVM-compatible Polygon Chain/PoS; 137 / 80002 Amoy; current RPC docs use POL.
Chain ID	Use it to select network-specific configuration and prevent wrong-network actions.
Bridge	Cross-chain validation + token identity + timing + admin risk.
Solana account	Fundamental state unit identified by 32-byte address.
Solana program	Executable sBPF account; mutable app state is in separate data accounts.
Instruction	Program ID + account list + instruction data.
PDA	Deterministic off-curve address derived from seeds + program ID; no private key.
SPL Mint	Defines token identity/shared mint state.
Token account / ATA	Holds token units for an owner and mint.
Anchor	Rust framework with account constraints and IDL generation.

Final mental model: EVM multi-chain = reuse code, re-validate environment. Solana = redesign around programs, accounts and explicit account constraints.

36. Instructor Whiteboard Flow - 60 Minutes

EVM PORTABILITY

Solidity -> compile/test -> RPC + chainId -> deploy -> verify -> address registry

L2 CONTEXT

User tx -> L2 execution -> commitments/data -> Ethereum settlement

CROSS-CHAIN

Chain A -> bridge/message validation -> Chain B

SOLANA RESET

Program != contract storage

Program -> instruction -> explicit accounts -> runtime -> state accounts

SPL TOKEN

Mint -> ATA/token accounts -> Token Program -> authority signatures

PDA

seeds + program ID -> deterministic address -> program-controlled state/authority

Teaching prompts

1. Ask: "Is Base a new Solidity language?" No - it is an EVM-compatible L2.
2. Ask: "Can I use the Ethereum USDC address on every chain?" No - token identity is chain-specific.
3. Ask: "Where is a Solana program's mutable state?" In separate accounts.
4. Ask: "Does a PDA have a private key?" No.
5. Ask: "What is the first job-scoping question?" EVM-compatible destination or different runtime?

37. Primary Technical References

Use official documentation as the source of truth before production work. The URLs below were checked for this August 2026 lecture.

Base - Connecting to Base: <https://docs.base.org/base-chain/quickstart/connecting-to-base>

Base - RPC Overview: <https://docs.base.org/base-chain/api-reference/rpc-overview>

Arbitrum - Get started / Solidity support: <https://docs.arbitrum.io/>

Arbitrum - Node providers / chain info: <https://docs.arbitrum.io/arbitrum-essentials/reference/node-providers>

Optimism - OP Stack documentation: <https://docs.optimism.io/>

Optimism - Differences from Ethereum: <https://docs.optimism.io/op-stack/protocol/differences>

BNB Smart Chain - Overview: <https://docs.bnbchain.org/bnb-smart-chain/overview/>

BNB Smart Chain - Wallet configuration: <https://docs.bnbchain.org/bnb-smart-chain/developers/wallet-configuration/>

Polygon - Polygon Chain / PoS: <https://docs.polygon.technology/pos>

Polygon - RPC endpoints: <https://docs.polygon.technology/pos/reference/rpc-endpoints>

Solana - Accounts: <https://solana.com/docs/core/accounts>

Solana - Programs: <https://solana.com/docs/core/programs>

Solana - Program Derived Addresses: <https://solana.com/docs/core/pda>

Solana - Tokens: <https://solana.com/docs/tokens>

Solana - Clusters and RPC endpoints: <https://solana.com/docs/references/clusters>

Anchor - Framework basics: <https://www.anchor-lang.com/docs/basics>

Anchor - Installation / testing: <https://www.anchor-lang.com/docs/installation>

Next lecture - Day 18: Real Client Blockchain Project: requirements -> architecture -> smart contracts -> tests -> frontend/backend -> testnet delivery -> documentation. Day 18 combines the full course into a client-style implementation workflow.